FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

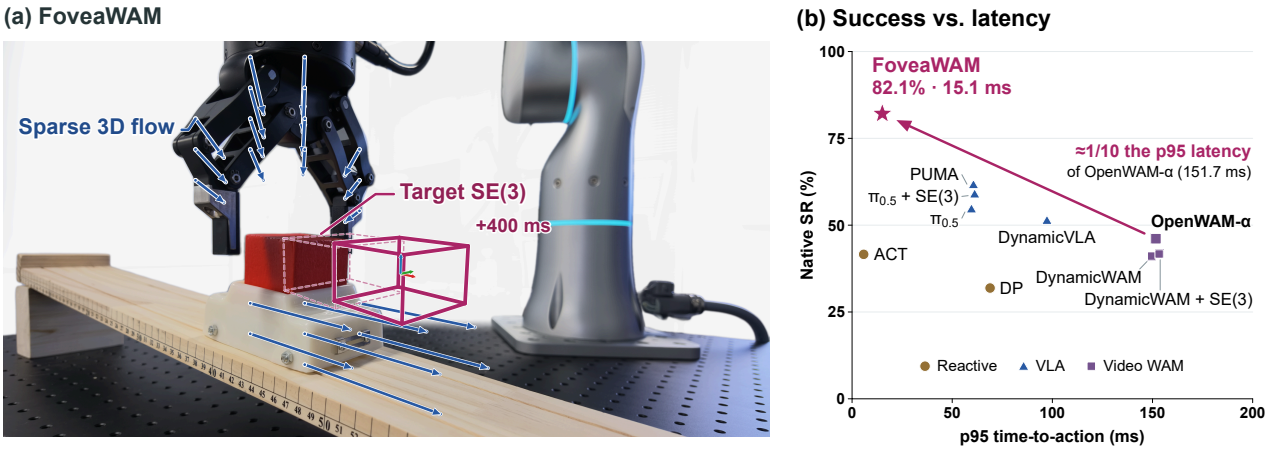


Figure 1. **FoveaWAM allocates prediction precision by role.** (a) It forecasts a precise SE(3) trajectory for the target and sparse 3D flow for its surroundings. (b) Success rate (SR) at native latency versus p95 time-to-action on OnTimeBench: FoveaWAM has the highest SR at about one tenth the latency of video world–action models.

Abstract

To manipulate a moving object, a policy must predict the scene at the moment its action executes, yet every millisecond spent predicting pushes that moment further away. Existing policies either forecast the whole scene at uniform fidelity and act late, or act fast by ignoring everything but the target. Our key insight is that what a policy must predict decouples by role: only the target's pose, which decides contact, needs millimeter precision, while the surroundings matter only through the clearance they leave and the motion they impart. We present FoveaWAM, a native 4D world–action model that allocates precision by role: it models the target's past and future motion as a metric SE(3) trajectory and the surroundings as sparse 3D flow, and relies on the current frame for everything static. This compact representation lets FoveaWAM act in 15.1 ms on one RTX 5090, about $10\times$ *faster than video world–action baselines, giving the scene little time to move. To evaluate each policy under its own latency, we introduce OnTimeBench, a dynamic manipulation benchmark in which the scene keeps moving while the policy computes. FoveaWAM reaches 82.1% success on OnTimeBench, 20.7 points above the strongest baseline, and also leads on DOMINO and real-robot tasks.*

1. Introduction

Robot manipulation policies have advanced rapidly in scenes that wait for them [5, 10, 49]: however long a policy computes, the scene it observed is still the scene it acts on. A moving object breaks this assumption. A ball rolling down a ramp at about 0.9 m/s travels 18 cm during a 200 ms inference, while contact tolerates only millimeters. The policy must therefore act on a prediction of where the scene will be when its action executes. But prediction is itself computation, and it lengthens the very interval it must cover: the more a policy predicts, the later it acts, and the further ahead it must predict. Existing dynamic policies settle this tension at one of two ends (Fig. 2). Some forecast the whole scene at uniform fidelity, as future video, latents, or 3D dynamics [29, 30, 44, 49, 54, 57]; they see everything but act late. Others forecast only the manipulated object [14, 17]; they act sooner but miss the surroundings that block or redirect it. Both tie precision to coverage: whatever is predicted is predicted in full, so knowing more of the scene means acting later. We ask instead: *what must a policy know about a moving scene, and how precisely, to make contact in time?*

Our key insight is that the answer decouples by role (Fig. 1). The target decides contact, so its pose at the moment of execution must be known to millimeter precision. The

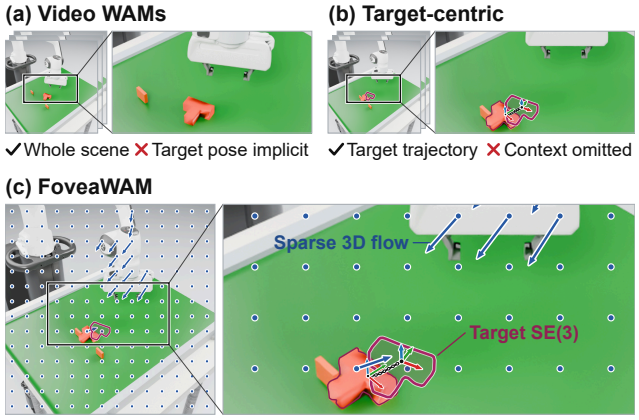


Figure 2. **Where forecast precision goes.** (a) Video world–action models (WAMs) forecast the whole scene and leave the target pose implicit [44, 49]. (b) Target-centric predictors forecast only the target's trajectory [14, 17]. (c) FoveaWAM uses one current frame and the target's pose history to forecast a precise SE(3) target trajectory and sparse 3D scene flow. The stacks in (a) and (b) show future frames and observation history, respectively. Panels (b) and (c) overlay future motion on the current frame.

surroundings matter only through the clearance they leave and the motion they impart to the target, so their motion need only be known coarsely. Everything static is already in the current frame and needs no prediction at all. What must be known precisely is therefore one object's trajectory, not the scene, and a policy that spends its precision there can be exact where contact is decided and still act in time. Biological vision makes the same allocation: the fovea resolves fine detail only where gaze lands, the periphery stays coarse yet alert to motion, and gaze runs ahead of the hand to where contact will occur [12, 19, 23, 33, 38].

We build this allocation into **FoveaWAM**, a native 4D world–action model whose representation follows the three roles (Fig. 2c). The target is described by a metric SE(3) trajectory: a short pose history supplies its past motion in place of past image frames, and the model learns to forecast its continuation. The surroundings are described by sparse 3D flow, and everything static by the current frame alone. Because motion is represented by one trajectory and a sparse flow rather than by frames, FoveaWAM acts in 15.1 ms on one RTX 5090, about 10× faster than video world–action models, leaving the scene little time to move.

Whether a policy acts in time can be measured only where latency has a cost. We therefore introduce **OnTimeBench**, a benchmark of 14 dynamic manipulation tasks in which the world never pauses: the scene keeps moving for as long as each policy computes, so every method is evaluated at its native latency. On OnTimeBench, FoveaWAM reaches 82.1% success, 20.7 points above the strongest baseline, and it also leads on DOMINO [14] and on real-robot tasks. Ablations verify the allocation: removing the target's pose history, its SE(3) forecast, or the sparse flow of its surroundings each lowers success. Evaluating at native latency also changes the ranking of existing policies: the strongest baseline with the world paused loses 17.3 points and its lead once the world keeps moving.

Our contributions are threefold:

- We identify the tension at the core of dynamic manipulation, that predicting a moving scene takes time in which the scene moves further, and show that allocating precision by role resolves it: millimeter precision for the target's pose at execution time, coarse motion for the surroundings, and no prediction for the static scene. Ablations verify each part of this allocation.
- We present **FoveaWAM**, a native 4D world–action model that realizes this allocation with a metric SE(3) target trajectory and sparse 3D scene flow. It acts in 15.1 ms, about 10× faster than video world–action models, and leads the strongest baselines by 20.7 points on OnTimeBench and 19.9 points on DOMINO, with further gains on real robot.
- We introduce **OnTimeBench**, a benchmark of 14 dynamic manipulation tasks in which the world never pauses and every policy is evaluated at its native latency, exposing costs of latency that paused evaluation hides.

2. Related Work

Dynamic manipulation. Acting on moving objects, long handled by engineered tracking and interception [1–3, 32, 36], is now learned from data [9, 25, 47]. Learned policies take motion from image-space cues, such as optical flow in PUMA [14] and DynamicWAM [29], or act on test-time forecasts of the scene [30, 40, 54, 57]. FoveaWAM instead reads the metric SE(3) pose history and observes only the current frame. Execution schemes [6, 27, 35, 41, 47] change when actions are applied, not what the policy represents, and are complementary. DOMINO [14] and PhysMani-Bench [54] provide dynamic tasks but by default pause the simulation until the policy returns. OnTimeBench advances it during inference, so each policy is evaluated at its own latency.

Forecasting in world–action models. World–action models (WAMs) generate future video and actions in one network [24, 44, 49, 61], building on video prediction as a policy objective [42, 46]. Because video is costly to generate, recent work forecasts cheaper targets, such as features, flow, object slots, or compact latents [28, 39, 50, 52, 56, 60], and some use the forecast only in training [14, 53, 58]. In all of them the forecast lives in pixel, feature, or latent space, and the target's future pose stays implicit. FoveaWAM keeps the training-only use but forecasts a metric 4D state whose precision follows role, so the target's future pose is supervised explicitly.

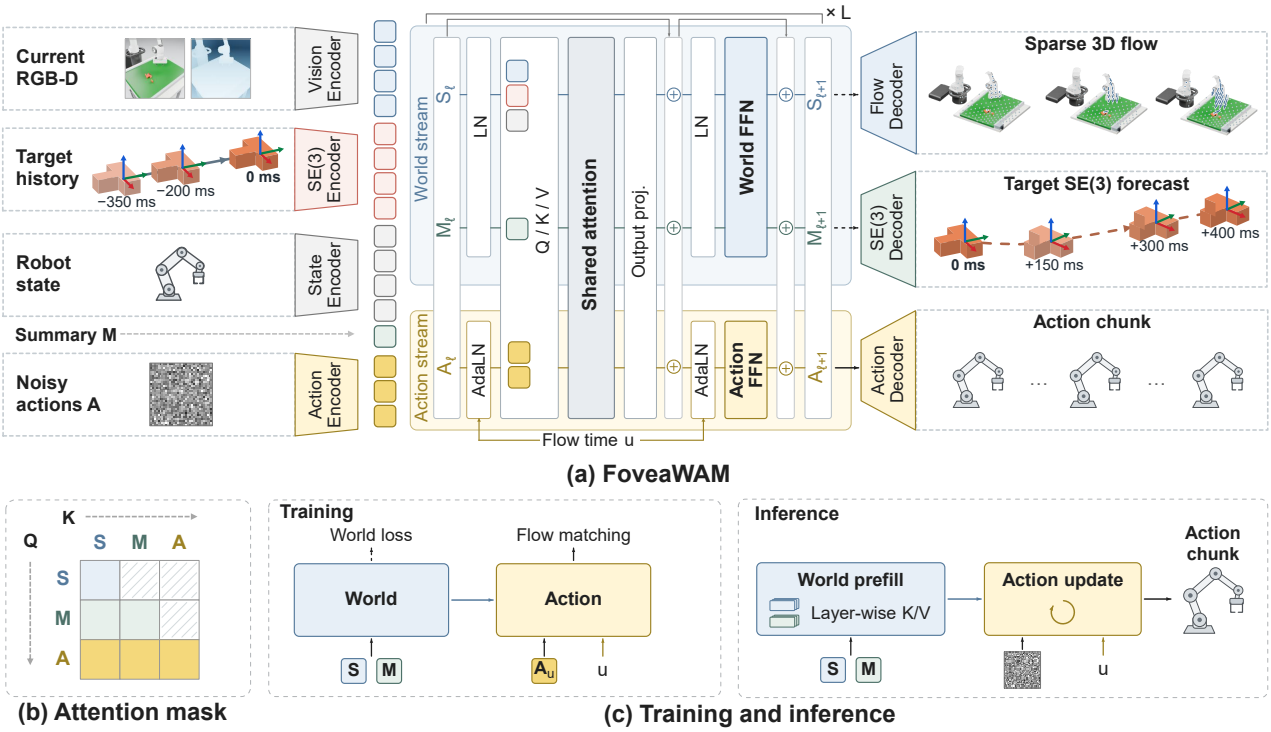


Figure 3. **Overview of FoveaWAM.** (a) Architecture. The current RGB-D frame and the target and robot histories form scene tokens S, alongside summary tokens M and noisy action tokens A. All tokens share masked attention, followed by separate world and action FFNs. The sparse 3D flow and the target SE(3) trajectory are predicted from S and M. (b) Attention mask. Rows are queries and columns are keys. Hatched cells are masked. (c) Training and inference. The world and action streams are trained jointly. At inference, the world stream runs once and the prediction heads are skipped.

Object-centric and geometric motion representations. Metric 3D inputs improve precision and sample efficiency [34, 55], and motion has been represented by point tracks [4, 45, 48], 3D flow [15, 18, 51, 54], object keypoints or SE(3) trajectories [17, 31, 37], and rigid-body decompositions [7, 8]. Each applies one representation to the whole scene, or models the target alone [14, 17]. DOCIR separates robot, target, and obstacles, but only as input encodings [13], and foveated robot perception allocates image resolution around a gaze point [11, 20, 22]. FoveaWAM allocates the precision of what is predicted: coarse sparse flow for the scene and a metric SE(3) trajectory for the target.

3. FoveaWAM

We present FoveaWAM, a native 4D world–action model that allocates precision by role on both sides of the present (Fig. 3). It encodes the past (Sec. 3.2) and predicts the future (Sec. 3.3) with the same split: a metric SE(3) trajectory for the target, coarse 3D geometry for its surroundings, and the current frame alone for everything static. An action stream reads this representation to generate time-indexed commands (Sec. 3.4).

3.1. Problem Formulation

We study manipulation in scenes that keep moving while the policy computes. At query t, a policy π learned from demonstrations reads an RGB image I_t and a depth map D_t captured at time τ_t, together with histories $\mathcal{T}_t$ of the target pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$ and $\mathcal{R}_t$ of the robot state, all in the robot base frame, and outputs a chunk of H commands

$$(\mathbf{a}_{t,0}, \ldots, \mathbf{a}_{t,H-1}) \sim \pi\big(\cdot \mid I_t, D_t, \mathcal{T}_t, \mathcal{R}_t\big), \qquad (1)$$

where command j is meant for time $\tau_t + j\Delta_a$, with control period Δ_a.

Each task assigns the scene three roles. The *target* is the object whose pose the robot must match within a contact or containment tolerance. The *context* is the part of the surroundings that matters: obstacles, which the robot must clear, and drivers, which move the target. Everything else is *static*. The chunk becomes available only after inference, so each executed command must account for the target and the context at its execution time rather than at τ_t.

3.2. Role-Aware Dynamics Encoding

What a policy must know about the past decouples by role. The static scene has no history worth encoding: the current frame already gives its layout. The context acts through the clearance it leaves, which is current geometry, and the motion it imparts, which shows in how the target has moved. Only the target's motion must be known precisely, and that motion is a rigid trajectory in SE(3). FoveaWAM therefore observes a single RGB-D frame for the scene and a short metric pose history for the target. Policies that take motion from stacked frames [10, 47] or optical-flow histories [14, 29] extract it densely over the whole image at every past step, and the target's velocity still remains implicit in appearance. FoveaWAM receives that velocity in physical units from a few low-dimensional tokens. This encoding keeps the model light enough to act within one control period while keeping the target's motion metric.

Static scene from the current frame. FoveaWAM anchors each patch token at a metric 3D point, so that the observation shares the base frame of the predictions and the actions. A frozen MAE ViT-B/16 [16] turns the RGB image into 196 patch tokens on a 14×14 grid, and the depth map back-projects the center of patch i to a point $\mathbf{x}_i \in \mathbb{R}^3$ in the base frame. A linear embedding of $\mathbf{x}_i$ is added to the patch token, and $\mathbf{x}_i$ drives 3D rotary position embeddings of the patch queries and keys, so that attention between patches depends on their relative 3D positions. One frame thus gives the scene layout and its clearances as metric geometry. The points $\mathbf{x}_i$ are also the source points of the scene flow (Sec. 3.3).

Target motion from the pose history. The target's recent motion enters as a metric SE(3) state. The history spans the last 350 ms in 8 samples at 20 Hz; its 8 tokens hold the current target pose and the seven motions between consecutive samples, each with its translation, relative rotation, time interval, and finite-difference velocity and acceleration in physical units. The velocity and acceleration that decide where the target will be thus enter the model directly, and a driver such as a conveyor or a ramp is captured through the motion it has imparted to the target. Poses come from the simulator in simulation and from an online RGB-D tracker on the robot.

World stream. The patch tokens, the target tokens, and 8 robot tokens with the end-effector states at the times of the target history form the 212 scene tokens. Together with 16 learnable summary tokens that aggregate them, they make up the world stream. The entire motion history thus costs sixteen low-dimensional tokens, whereas each additional frame would cost 196.

3.3. Role-Aware 4D Future Prediction

Every command lands in a later scene than the one observed, so the representation must anticipate the scene at the execution times of the commands, and the precision this requires again follows role. Contact is decided by the target's pose, so FoveaWAM predicts the target's full SE(3) trajectory. The context matters through clearance and imparted motion, so FoveaWAM predicts the motion of the scene only coarsely, as a sparse 3D flow on the patch grid. Both predictions are expressed in the base frame as changes from the current state at the times $\tau_t + \delta_h$, with $\delta_h = h \cdot 50\,\text{ms}$ for $h = 1, \dots, 8$; together with the current state, these horizons cover the execution times of all commands in the chunk. Both are linear readouts of tokens that the action stream already reads: they shape the representation during training and are skipped at inference, so prediction adds no latency.

Sparse 3D scene flow. At the resolution of the patch grid, the flow covers every surface that the patches see: the robot, the target, and the context, whose clearance and motion the commands must account for. For each of the 196 points $\mathbf{x}_i$ and each horizon h, a linear head on the final state of patch token i predicts the point's base-frame displacement $\hat{F}_{ih} \in \mathbb{R}^3$ and a visibility logit $\hat{o}_{ih}$, trained with

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h)\in\mathcal{V}_F} \Big[\tfrac{1}{3} \big\| (\hat{F}_{ih} - F_{ih})/s \big\|^2 + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \tag{2}$$

where F_{ih} is the displacement of the surface point at $\mathbf{x}_i$, o_{ih} its visibility, s a fixed scale fitted on the training labels, and $\mathcal{V}_F$ the set of valid labels.

Target SE(3) trajectory. The flow samples the target only at the few patches it covers, too sparse to determine its rotation. A dedicated linear head on the mean of the final summary states therefore predicts its rigid motion directly: for each horizon h, the target's displacement $\hat{\mathbf{d}}_h$ and a rotation increment $\hat{\boldsymbol{\phi}}_h$, trained with

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h\in\mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \Big], \tag{3}$$

where $\mathbf{d}_h = \mathbf{p}(\tau_t + \delta_h) - \mathbf{p}(\tau_t)$ and $R_h = R(\tau_t + \delta_h)$ are the true displacement and orientation, $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^{\wedge}) R(\tau_t)$, $d_{\mathrm{SO}(3)}$ is the geodesic distance, and $\mathcal{V}$ is the set of horizons with valid labels. Contact demands the rotation term: a rotation error θ displaces a contact point at distance r from the target's origin by up to $2r\sin(\theta/2)$, nearly 9 mm for 5° at $r = 10\,\text{cm}$.

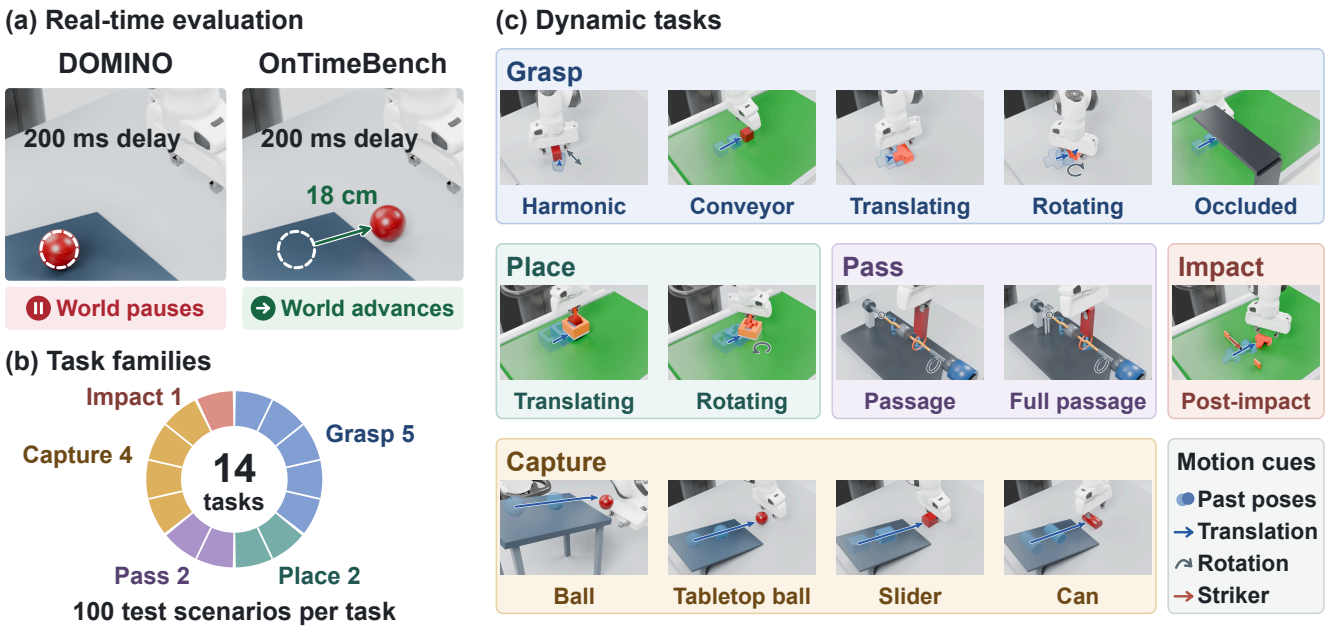


Figure 4. **Overview of OnTimeBench.** OnTimeBench evaluates each policy at its native latency on 14 dynamic tasks in five families, with 1,400 held-out test scenes. (a) During a 200 ms inference, the default DOMINO evaluator [14] pauses the world, while OnTimeBench keeps it moving. (b) Tasks per family. (c) The 14 tasks, with motion cues overlaid.

3.4. World–Action Learning and Inference

Action stream. The action stream generates the chunk by conditional flow matching [26]. It holds $H = 8$ tokens, each embedding one noisy command: an end-effector translation increment, a rotation increment, and a gripper command. In every layer, one attention spans both streams with shared projections, followed by separate feed-forward networks; world tokens use layer normalization, and action tokens use adaptive layer normalization conditioned on the flow time. With S, M, and A denoting the scene, summary, and action tokens, the attention mask allows $S \to S$, $M \to \{S, M\}$, and $A \to \{S, M, A\}$, where $X \to \mathcal{Y}$ means that tokens in X attend to tokens in $\mathcal{Y}$. The action stream thus reads, in every layer, both the patch states shaped by the scene flow and the summary states shaped by the target trajectory, while the world stream never reads the actions.

Training objective. For a chunk $\mathbf{a}_1$ normalized with training statistics, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and flow time $u \sim \mathcal{U}[0, 1]$, the action stream v_ψ regresses the flow-matching velocity

$$\mathcal{L}_A = \mathbb{E}\,\mathrm{MSE}\big(v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t),\, \mathbf{a}_1 - \mathbf{a}_0\big), \tag{4}$$

where $\mathbf{a}_u = (1 - u)\mathbf{a}_0 + u\mathbf{a}_1$, $\mathcal{S}_t$ denotes the inputs in Eq. (1), and the MSE averages over valid commands and action dimensions. The world and action streams are trained jointly with

$$\mathcal{L} = \mathcal{L}_A + \lambda_F \mathcal{L}_F + \lambda_P \mathcal{L}_P + \lambda_{\text{aux}} \mathcal{L}_{\text{aux}}, \tag{5}$$

where $\mathcal{L}_F$ and $\mathcal{L}_P$ are the flow and pose losses of Eqs. (2) and (3), and $\mathcal{L}_{\text{aux}}$ sums auxiliary appearance losses.

Inference. At deployment, FoveaWAM skips both prediction heads [24, 53, 56]. Because world tokens attend neither to the actions nor to the flow time, the world stream runs once per query, and the action stream integrates 5 Euler steps on the cached keys and values of the scene and summary tokens. One query takes 15.1 ms at p95 on one RTX 5090, within a single control period. Commands are indexed by time from capture, so the policy needs no latency input: whenever a chunk arrives, execution starts from the command indexed to that time (Sec. 4.3). Further implementation details are in the supplementary material.

4. The OnTimeBench Benchmark

4.1. Overview

A dynamic benchmark must charge each policy for the time it takes to compute. When the simulation advances only after the policy has returned its actions, as in the default evaluators of DOMINO [14] and PhysMani-Bench [54], a slow policy is scored as if it were instant: the evaluation measures what a policy predicts but not whether the prediction arrives in time, which is what decides success on a real robot.

To evaluate whether a policy can act on a moving scene in time, we introduce OnTimeBench, a benchmark of 14 dynamic manipulation tasks in which the world never pauses. Observation, scene motion, and action execution share one physical clock, so the scene keeps moving for as long as each policy computes. Fig. 4(a) shows what this changes: during a 200 ms inference, a rolling ball waits at its observed position when the world is paused, but in OnTimeBench it has rolled 18 cm, so an action aimed at the observed position misses. OnTimeBench covers five task families (Fig. 4(b)): *Grasp*, grasping a target under harmonic, conveyor, translating, rotating, or occluded motion; *Place*, placing a held object into a translating or rotating receptacle; *Pass*, passing a wand along a rotating wire; *Capture*, capturing a rolling or sliding object; and *Impact*, grasping a part after an impact. Fig. 4(c) shows all 14 tasks. Together they vary what decides success: the target's speed, from 0.03 to 1.14 m/s, its rotation, the clearance of obstacles, and the motion imparted by drivers such as conveyors, ramps, and strikers.

4.2. Construction

OnTimeBench is built in MuJoCo and robosuite [43, 62] with a Franka Panda arm controlled at 20 Hz. Each scene samples the target's initial state and motion parameters within task-specific ranges. A task-specific expert with privileged simulator state provides 800 demonstrations per task, 11,200 in total. All results are reported on 1,400 held-out test scenes, 100 per task, shared by all methods; configurations are selected on separate development scenes.

4.3. Evaluation Protocol

Native latency. During each query, physics advances by the method's own inference latency, drawn from its latency distribution measured at batch size 1 on one RTX 5090. All methods share one executor, which hands over each chunk at the first control-period boundary after inference completes and executes it from the command indexed to that boundary.

Settings and metrics. We evaluate three settings: *Native*, at each method's own latency; *+150 ms*, with 150 ms added to every method; and *Paused*, with the world paused during inference. We report the task-balanced success rate (SR) and the p95 time-to-action, the time from capturing an observation until its first command is available. The full protocol is in the supplementary material.

5. Experiments

5.1. Implementation Details

Baselines. We compare with the reactive policies ACT [59] and Diffusion Policy (DP) [10], the vision–language–action model (VLA) $\pi_{0.5}$ [5], the dynamic VLAs DynamicVLA [47] and PUMA [14], the WAM OpenWAM-α [44], and the dynamic WAM DynamicWAM [29]. Each baseline keeps its official architecture and training recipe and is trained on the same demonstrations. To isolate the effect of target-motion input, $\pi_{0.5}$ and DynamicWAM also run with our SE(3) history tokens (+SE(3)).

Table 1. **Quantitative comparison on OnTimeBench.** SR (%) at each method's native latency, with 150 ms added, and with the world paused during inference. p95: time-to-action on one RTX 5090. Baseline rollouts use only the main view. History: inputs that carry past motion. (off): caching and compilation disabled. Best and second best are highlighted.

Method	History			SR (%) ↑			p95 (ms) ↓
	RGB	Flow	SE(3)	Native	+150 ms	Paused	
ACT [59]				41.6	24.6	40.6	**5.8**
DP [10]	✓			31.9	16.1	32.0	68.9
$\pi_{0.5}$ [5]				54.4	28.1	66.6	59.6
$\pi_{0.5}$+SE(3)			✓	58.7	28.0	76.0	61.2
DynamicVLA [47]	✓			51.1	23.5	54.9	97.3
PUMA [14]		✓		61.4	31.1	53.1	60.6
OpenWAM-α [44]				46.1	24.8	58.7	151.7
OpenWAM-α (off)				15.8	10.4	58.4	466.3
DynamicWAM [29]		✓		41.1	20.6	49.6	149.6
DynamicWAM+SE(3)		✓	✓	41.7	24.9	64.0	153.3
FoveaWAM			✓	**82.1**	**34.1**	**90.3**	15.1

Model and training. The transformer of FoveaWAM has 12 layers of width 1024 with 16 heads and 271.8M trainable parameters on top of the frozen MAE ViT-B/16 encoder. On OnTimeBench, it is trained from random initialization with AdamW for 10 epochs (25,870 updates). Inference runs 5 Euler steps in compiled BF16. FoveaWAM observes one agent-view RGB-D frame and the target's SE(3) pose history from the simulator.

5.2. Dynamic Manipulation Results

Results on OnTimeBench. As shown in Table 1, FoveaWAM achieves 82.1% SR at native latency, 20.7 points above the strongest baseline, PUMA (61.4%), and the highest SR in all three settings. It has the highest or tied-highest SR on 13 of the 14 tasks (Table S4), with the largest margins where a moving receptacle or the target's orientation decides success: 30 and 55 points on placement into translating and rotating receptacles, and 28 points on full wire passage. Precise target motion and low latency are both required for this result. Given the same SE(3) history, $\pi_{0.5}$ and DynamicWAM reach only 58.7% and 41.7%, and ACT, the only faster method, reaches 41.6%.

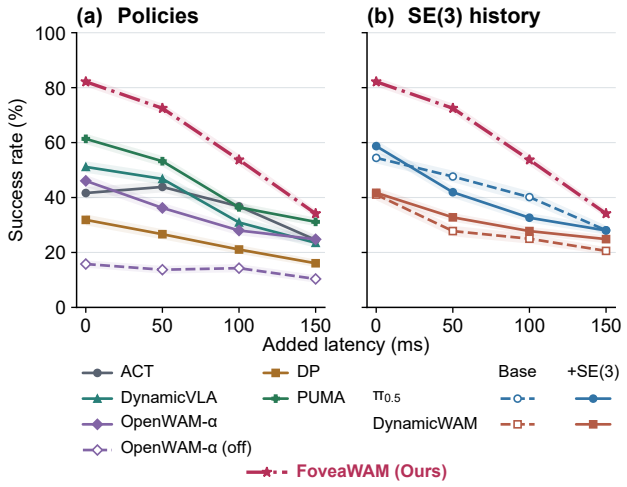


Figure 5. **Success under added latency.** SR on OnTimeBench as delay is added to each method's native latency. (a) All methods. (off): caching and compilation disabled. (b) Base versus +SE(3) variants. Bands: 95% intervals.

Effect of inference latency. Latency alone can decide success. The two OpenWAM-α deployments run the same policy and agree with the world paused (58.7% and 58.4%), yet at native latency the one with a p95 of 151.7 ms reaches 46.1% and the one with 466.3 ms reaches 15.8%. Latency also erodes the value of motion input: the SE(3) history raises $\pi_{0.5}$ and DynamicWAM by 9.4 and 14.4 points with the world paused but by only 4.3 and 0.6 at native latency. Paused evaluation therefore overestimates slow policies and misorders methods: $\pi_{0.5}$+SE(3), the strongest baseline with the world paused (76.0%), loses 17.3 points at native latency and falls behind PUMA. FoveaWAM leads in both settings and remains the best method at every added delay (Fig. 5). With 150 ms added, every method falls to 35% or below, since the remaining commands of each chunk must cover a longer open-loop interval; FoveaWAM stays ahead, but its margin over PUMA narrows from 20.7 to 3.0 points.

Results on DOMINO. To evaluate FoveaWAM beyond OnTimeBench, we train FoveaWAM, $\pi_{0.5}$, DynamicWAM, and PUMA on the 24 single-arm Franka tasks of DOMINO [14] with its demonstrations and its native evaluator. As shown in Table 2, FoveaWAM achieves the highest overall SR, 46.8%, 19.9 points above the strongest baseline, DynamicWAM (26.9%), and the highest manipulation score. This evaluator pauses physics during inference, so the lead reflects the quality of the role-aware representation rather than its speed. The margin is largest on the 16 Track tasks, 55.5% against 31.9% for DynamicWAM.

Table 2. **Quantitative comparison on DOMINO** [14]. SR (%) on its 24 single-arm tasks, 8 Intercept and 16 Track. MS: manipulation score. Best and second best are highlighted.

Method	Intercept	Track	All ↑	MS ↑
$\pi_{0.5}$	23.8	8.1	13.3	27.8
DynamicWAM	16.8	31.9	26.9	41.4
PUMA	**36.3**	12.2	20.2	35.4
FoveaWAM	29.4	**55.5**	**46.8**	**62.0**

Table 3. **Ablation results on OnTimeBench.** SR (%) of FoveaWAM and its change for each variant (Δ, points; red: drop, green: gain). Rot./Ctx.: rotation-sensitive and context-active tasks.

	Success rate, all tasks		Groups (Native)	
	Native	+150 ms	Rot.	Ctx.
FoveaWAM	**82.1**	**34.1**	**80.4**	**82.7**
Precision allocation				
Uniform coarse flow	−4.5	−0.5	−8.6	−4.6
Uniform fine flow	+0.8	−0.9	−3.4	−2.6
w/o flow	−4.2	−1.4	−11.7	−3.9
Target motion history				
w/o history	−19.6	+0.2	−24.0	−2.3
Future supervision				
w/o future	−4.3	−3.4	−12.6	−2.1
w/o $\mathcal{L}_{\text{aux}}$	−2.6	−1.9	−11.6	+0.1

5.3. Ablation Study

We ablate each part of the role-aware design on OnTimeBench (Table 3) and report SR at native latency, overall and on the rotation-sensitive (Rot.) and context-active (Ctx.) task groups (Table S2). All variants except *w/o history* change only the training supervision, so they run at the same latency as FoveaWAM.

Effect of role-aware prediction. Role-aware prediction gives the target a precision that uniform prediction does not. Replacing the SE(3) trajectory with uniform flow at the same density reduces SR by 4.5 points overall and by 8.6 on rotation-sensitive tasks. Raising the uniform flow to a 56×56 grid ($16\times$ as many points) recovers the overall SR (+0.8) but still trails by 3.4 points on rotation-sensitive tasks and by 2.6 on context-active tasks. Removing the sparse flow instead reduces SR by 4.2 points overall, by 3.9 on context-active tasks, and by 11.7 on rotation-sensitive tasks. The two predictions are thus complementary: one target trajectory and 196 flow points match 3,136 uniform flow points overall and surpass them where rotation and context matter.

Effect of target motion history. Removing the pose history while keeping the current target pose reduces SR by 19.6 points overall and by 24.0 on rotation-sensitive tasks, the largest drop among all variants. A single pose does not

show how the target moves; the metric history supplies the motion that the commands must anticipate.

Effect of future supervision. Supervising only the current target state instead of its future reduces SR by 4.3 points overall and by 12.6 on rotation-sensitive tasks, although the prediction heads never run at deployment: the forecast acts through the representation it shapes, not through its output. Removing the auxiliary objectives $\mathcal{L}_{\text{aux}}$ reduces SR by 2.6 points overall and by 11.6 on rotation-sensitive tasks.

5.4. Real-Robot Evaluation

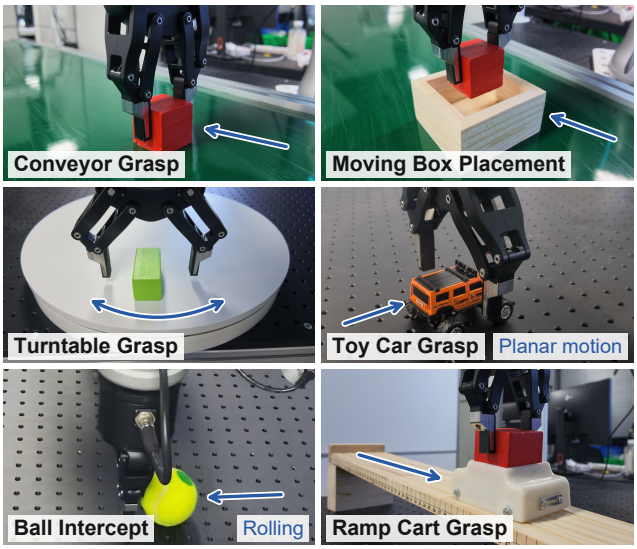


Figure 6. **Real-robot tasks.** Arrows show the motion of the target or the moving box before contact.

Setup. We evaluate on six dynamic tasks with a Flexiv Rizon 4s arm with a GN01 gripper (Fig. 6 and Table S8). All methods are trained on a shared set of 600 teleoperated demonstrations and evaluated for 20 trials per task. FoveaWAM starts from a DROID-pretrained base [21] and uses RGB-D and online target pose estimates from one fixed camera. The baselines, $\pi_{0.5}$, ACT, and DynamicWAM, use RGB from the fixed and wrist cameras.

Table 4. **Real-robot results.** SR (%) over 20 trials per task. p95: end-to-end time-to-action (ms). Best is highlighted.

Method	Belt grasp	Box place	Turntable	Toy car grasp	Tennis ball	Ramp grasp	All ↑	p95 (ms) ↓
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
ACT	—	—	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	37.5	—

Results. As shown in Table 4, FoveaWAM succeeds in 37.5% of all trials, against [XX.X]% for [the strongest baseline], with the largest margin on [tasks]. Its p95 latency, including tracking, is [XX] ms, against [XX] ms for [name]. [One trajectory-verified contact example: how the target's pose, rotation, or context explains a success of FoveaWAM or a failure of a baseline.]

6. Conclusion

We introduce FoveaWAM, a native 4D world–action model that addresses a tension at the core of dynamic manipulation: predicting a moving scene takes time, and that time lets the scene move further. By allocating precision by role, with a metric SE(3) trajectory for the target, sparse 3D flow for its surroundings, and the current frame for everything static, FoveaWAM is precise where contact is decided and acts in 15.1 ms. Together with OnTimeBench, which evaluates every policy at its native latency, it offers a model and a benchmark for building and evaluating policies that act on time.

LLM Usage Statement

We used OpenAI Codex to assist with literature review, code implementation and debugging, experiment execution, and language editing of the manuscript. Codex also assisted in developing plotting code and refining figure and manuscript layouts. The authors reviewed AI-assisted content, checked references against original sources, and validated the implementation through testing and experiments. The authors take full responsibility for the final manuscript, experimental results, and conclusions.

References

[1] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[2] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.

[3] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[4] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 3

[5] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 1, 6

[6] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2

[7] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, pages 173–180, 2017. 3

[8] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 3

[9] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[10] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 4, 6

[11] Ian Chuang, Jinyu Zou, Andrew Lee, Dechen Gao, and Iman Soltani. Look, focus, act: Efficient and robust robot learning via human gaze and foveated vision transformers. In *IEEE International Conference on Robotics and Automation*, pages 18506–18513, 2026. 3

[12] Christine A. Curcio, Kenneth R. Sloan, Robert E. Kalina, and Anita E. Hendrickson. Human photoreceptor topography. *Journal of Comparative Neurology*, 292(4):497–523, 1990. 2

[13] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 15873–15879, 2025. 3

[14] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 1, 2, 3, 4, 5, 6, 7

[15] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *IEEE Robotics and Automation Letters*, 11(3):2466–2473, 2026. 3

[16] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 4

[17] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 1, 2, 3

[18] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 3

[19] Roland S. Johansson, Göran Westling, Anders Bäckström, and J. Randall Flanagan. Eye–hand coordination in object manipulation. *Journal of Neuroscience*, 21(17):6917–6932, 2001. 2

[20] Justin Kerr, Kush Hari, Ethan Weber, Chung Min Kim, Brent Yi, Tyler Bonnen, Ken Goldberg, and Angjoo Kanazawa. Eye, robot: Learning to look to act with a BC-RL perception-action loop. In *Proceedings of The 9th Conference on Robot Learning*, pages 3647–3664, 2025. 3

[21] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 8

[22] Heecheol Kim, Yoshiyuki Ohmura, and Yasuo Kuniyoshi. Gaze-based dual resolution deep imitation learning for high-precision dexterous robot manipulation. *IEEE Robotics and Automation Letters*, 6(2):1630–1637, 2021. 3

[23] Michael F. Land and Peter McLeod. From eye movements to actions: How batsmen hit the ball. *Nature Neuroscience*, 3 (12):1340–1345, 2000. 2

[24] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2, 5

[25] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi

Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[26] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 5

[27] Yuejiang Liu, Jubayer Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, pages 4594–4627, 2025. 2

[28] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 2

[29] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 4, 6

[30] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 1, 2

[31] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *Robotics Research: The 19th International Symposium ISRR*, pages 132–157. Springer, 2022. 3

[32] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[33] Ruth Rosenholtz. Capabilities and limitations of peripheral vision. *Annual Review of Vision Science*, 2:437–457, 2016. 2

[34] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799, 2023. 3

[35] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[36] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[37] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. In *European Conference on Computer Vision*, pages 437–453, 2026. 3

[38] Hans Strasburger, Ingo Rentschler, and Martin Jüttner. Peripheral vision and pattern recognition: A review. *Journal of Vision*, 11(5):13, 2011. 2

[39] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 115917–115933, 2026. 2

[40] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. In *Conference on Robot Learning*, 2026. Accepted for publication. 2

[41] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[42] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, pages 92033–92052, 2025. 2

[43] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 6

[44] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 6

[45] Chuan Wen, Xingyu Lin, John Ian Reyes So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[46] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, pages 10641–10662, 2024. 2

[47] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 4, 6

[48] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, pages 2475–2499, 2025. 3

[49] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[50] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 2

[51] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2025. 3

[52] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 2

[53] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 5

[54] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. In *European Conference on Computer Vision*, pages 20–39, 2026. 1, 2, 3, 5

[55] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[56] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xin-Qiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 2, 5

[57] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 1, 2

[58] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 2

[59] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 6

[60] Ruijie Zheng, Jing Wang, Scott Reed, et al. FLARE: Robot learning with implicit world modeling. In *Proceedings of The 9th Conference on Robot Learning*, pages 3952–3971. PMLR, 2025. 2

[61] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[62] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 6